Jacobus Langner

Staff-level Full Stack Engineer · Cape Town



jacobus@hack-cove.com • +27 78 629 4044 • GitHub: madangryscientist • LinkedIn

Staff-level full stack engineer with 18 years of production software: TypeScript/React front ends, and the Node, Python and C#/.NET services behind them, connected through messaging (SQS/SNS, RabbitMQ), on Kubernetes and Cloudflare. I started Hack Cove in April 2026 for contract work. At Cue, my current client, I build the voice product on both sides: the live call and transfer flows in React, and the Python service that integrates with the voice provider. At Zappi I owned the front-end guild and the shared UI platform, and built the Node service that carried Salesforce events to the rest of the platform.

EXPERIENCE

Founder / Contract Engineer — Hack Cove

April 2026 – Present

I started Hack Cove to do engineering contract work. Cue is my current client.

Cue

- Building the Voice Inbox and live call and transfer flows in the TypeScript/React app.
- I work on the calls inbox as a product surface: listings, inbound answer that opens the voice session and contact context, and realtime updates. The call UI is a floating modal with a dialer, mic and speaker selection, and recipient and contact search.
- Implementing cold and warm transfers, including a transfer picker with live directory and presence, and in-flight actions to pull a transfer back, complete it, or mute.
- Built the Python service that integrates the app with the voice service provider.

Hack Cove UI

- An open-source React design system on npm (@hackcove/ui plus theme packages), in an Nx and Bun monorepo with Storybook. Every theme reskins every component with no code change.
- Its reusable GitHub Action reviews UI pull requests in two layers: deterministic rules that fail on off-system styling, then an LLM review through OpenCode Zen against the published component reference (llms.txt).
  hack-cove.com/ui · github.com/madangryscientist/hackcove

PixelSlash

- A visual-regression tool. It has a Bun CLI and a pixel-diff engine in Rust compiled to WebAssembly. Its review app runs TanStack Start on Cloudflare Workers, with D1 through Drizzle, R2 for screenshots, GitHub sign-in, and AES-256-GCM encryption for the API keys it stores.
- AI review: a vision model judges each shot against the design system (local mlx or ollama, or a hosted model), and the server sends each changed region to an OpenAI-compatible vision endpoint on a Cloudflare Queue to explain the change, with retries and per-build token accounting. An MCP server lets coding agents such as Claude Code read builds and drive the tool.
  pixelslash.dev · github.com/madangryscientist/pixelslash

Tech: TypeScript, React, Python, Node/Bun, Rust/WebAssembly, Cloudflare Workers, D1, Drizzle

Senior Frontend Engineer — Zappi

April 2021 – April 2026

Working in a fully remote, distributed team maintaining shared UI component libraries and driving modernization of the frontend tooling stack. I owned and led the front-end guild for the last year there, driving technical direction and best practices across teams.

- Built a Node microservice that took events from Salesforce's messaging system and republished them on SQS/SNS so other services could consume them.
- Ran the front-end Node server: its build and the APIs I built to speed up page loads.
- Maintain a TypeScript monorepo of 60+ packages: shared UI components and tooling consumed by multiple frontend applications. Design library APIs balancing end-user simplicity with extensibility for internal consumers.
- Jest to Vitest: reduced test suite runtime from 90s to 58s (35% faster), with 60% faster transforms using SWC—100% pass rate across 1,126 tests
- Cypress to Playwright: reduced E2E test runtime from 30 minutes to 5 minutes (83% faster)
- Migrated library builds from CommonJS to ESM-only using tsdown, reducing build time from 120s to 5s (96% faster)
- Refactored microfrontend architecture to library-based imports, reducing bundle size from 41MB to 24MB (41% reduction)
- React 18 migration, resolving concurrent rendering issues across a large codebase
- Redux to React Query and Zustand, modernizing state management and data fetching patterns application-wide
- Managed NX monorepo upgrades (v21 → v22) with build caching optimization

Tech: TypeScript, Node.js, Next.js, React, Ruby, Storybook, Playwright, Cypress, Amazon SQS, Amazon SNS

Frontend-API Architect — FMI

June 2018 – March 2021

Architected the APIs and workflow engines (policy applications, claims, underwriting) for a life insurer moving off paper, in C# and TypeScript, with React/Next.js front ends. Built Kubernetes clusters from scratch and moved releases from monthly FTP drops to continuous delivery. Designed role-based access control and POPI Act compliance for medical and financial data.

Tech: Docker, TypeScript, C#, Next.js, React, Angular, Vue, Kubernetes

Software Development Team Lead — Wonga

August 2015 – May 2018

Led a full stack team building new products as C# microservices (CQRS, RabbitMQ), with ASP.NET Core Web API back ends and AngularJS front ends.

Starting as a back-end developer. Soon I moved into a technical lead developer position on the front-end as we migrated from our PHP site into an AngularJS site with ASP.NET Core Web API.

After the completion of that project I was promoted to team lead.

These efforts led to my team being awarded the CEO Recognition Award.

In mid-2017, I started doing more architectural work to help move away from our legacy system.

Tech: C#, Octopus Deploy, AngularJS, RabbitMQ

EARLIER ROLES

Software Developer — Sygnia Ltd **September 2014 – August 2015**

I worked as the bridge between web developers and the server-side developers, in an asset management and finance driven environment. I was often tasked with speeding up queries.

Software Developer — Property24 **August 2011 – August 2014**

As part of the Property24 team, I was involved with developing, maintaining and monitoring one of South Africa's most visited websites. This required software that scaled across multiple servers, with a heavy emphasis on caching. We upgraded from ASP Classic to ASP.NET to ASP MVC.

Software Developer — ACI Worldwide **January 2010 – August 2011**

Java on Windows and AIX, SQL Server and DB2. First year on JSP bug fixes, then 3rd-level support for PostCard; patches across multiple versions and platforms.

Software Developer — Business Intelligence **July 2009 – December 2009**

C# and ASP.NET. Most of the work was upgrading from ASP.NET 1.1 to ASP.NET 3.5; Ajax processing was the main focus, plus SQL Server 2008.

Software Developer — Axxess Dsl **September 2008 – July 2009**

C# Windows applications, Windows services, ASP.NET 2 websites and web services. Basic knowledge of MySQL.

EDUCATION

Bachelor of Science (BSc), Computer Science — NMMU, Port Elizabeth

2005–2008 • Automata Theory 83% • Linear Optimization 88% • Advanced Programming 71%/76%

SKILLS

Languages:	TypeScript, JavaScript, C#, Python, Ruby
Frameworks:	React, Next.js, Angular, Vue
Build Tools:	Vite, Vitest, NX, Webpack, tsdown, Biome
Testing:	Vitest, Playwright, Storybook, Chromatic
Infrastructure:	GitHub Actions, GitLab CI, Kubernetes, Docker
Databases:	PostgreSQL, SQL Server, MongoDB
Spoken:	English (Native), Afrikaans (Fluent)